

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»**

**Лабораторная работа №3
по курсу «Операционные системы»**

**Выполнил: П. Д. Косарим
Группа: М8О-207БВ-24
Преподаватель: Е. С. Миронов**

Москва, 2025

Условие

Цель работы: Приобретение практических навыков в:

- Освоение принципов работы с файловыми системами
- Обеспечение обмена данных между процессами посредством технологии «File mapping»

Задание: Составить и отладить программу на языке C++, осуществляющую работу с процессами и взаимодействие между ними в Linux. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или через отображаемые файлы (memory-mapped files). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы. Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процесс должны быть представлены разными программами. Результаты своей работы дочерний процесс пишет в созданный им файл. Пользователь вводит команды вида: «число число число<endline>». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип int. Количество чисел может быть произвольным.

Вариант: 1

Метод решения

1. Создаем разделяемый сегмент памяти для общения процессов.
2. Создаем дочерний процесс, через сегмент передаем данные для вычисления суммы.
3. Вычисляем сумму, записываем результат в файл, удаляем сегмент.

Описание программы

parent.c - Родительский процесс:

1. Создает и настраивает разделяемую память для обмена данными
2. Запрашивает у пользователя имя файла для сохранения результатов
3. Запускает дочерний процесс с помощью CloneProcess()
4. В цикле принимает числа от пользователя
5. Передает данные в разделяемую память и синхронизируется с дочерним процессом
6. Обрабатывает команду "stop" для завершения работы
7. Освобождает ресурсы и ожидает завершения дочернего процесса

child.c - Дочерний процесс:

1. Открывает файл для записи результатов (имя передается как аргумент)
2. Подключается к существующей разделяемой памяти
3. В цикле проверяет наличие новых данных от родителя
4. Валидирует входные данные (проверяет, что это числа)
5. Вычисляет сумму переданных чисел
6. Записывает числа и их сумму в файл
7. Выводит результат на экран
8. Завершает работу по сигналу от родителя

os.h - Кроссплатформенный интерфейс

1. Определяет общие типы данных (pipe_t, pid_t)
2. Объявляет функции для работы с процессами, pipes и разделяемой памятью
3. Обеспечивает переносимость между Windows и Linux системами

os_linux.c - Linux-реализация системных функций

1. Реализует все функции из os.h для Linux-систем
2. Использует системные вызовы: fork(), execv(), shm_open(), mmap()
3. Обеспечивает работу с процессами, pipes и разделяемой памятью в Linux

Используемые системные вызовы:

1. fork() (через CloneProcess()) - создание дочернего процесса
2. execv() (через Exec()) - загрузка новой программы
3. wait() (через WaitProcess()) - ожидание завершения дочернего процесса
4. sched_yield() - добровольная передача управления планировщику
5. shm_open() - создание/открытие объекта разделяемой памяти
6. ftruncate() - установка размера объекта разделяемой памяти
7. mmap() - отображение памяти в адресное пространство процесса
8. munmap() - удаление отображения памяти
9. shm_unlink() - удаление объекта разделяемой памяти

Результаты

При запуске программы, вводе имени файла и строки чисел, программа вычисляет сумму этих чисел и записывает результат в указанный файл, дублируя все информацию в консоль.

Пример работы:

ведите имя файла: 1.txt

ведите числа, для завершения введите 'stop'

child запущен

1 2 3 4 5 6 7

сумма: 28

1111 2222 333

сумма: 3666

11111111 555555555

сумма: 566666666

stop

child завершен

Выводы

Написали программу, которая создает дочерний процесс для вычисления суммы чисел.

Использование двух процессов позволяет разделять взаимодействие с пользователем и управление в родителе и обработку данных и вычисление результатов в дочернем

В программе использовали memory mapped files для передачи данных напрямую через память (не использовать системные вызовы), для простоты синхронизации процессов (через флаг ready), для прямого доступа к данным (без сериализации/десериализации). Memory mapped требует меньше ресурсов для работы в отличие от pipe в 1 пр.

Исходная программа

```
1 | #pragma once
2 |
3 | #ifdef _WIN32
4 |
5 | #else
6 | #include <errno.h>
7 | #include <fcntl.h>
8 | #include <stdio.h>
9 | #include <stdlib.h>
10 | #include <string.h>
11 | #include <sys/wait.h>
12 | #include <unistd.h>
13 | #include <sys/mman.h>
14 | #include <sys/stat.h>
15 |
16 | typedef int pipe_t;
17 | typedef int pid_t;
18 |
19 | typedef struct {
20 |     pid_t pid;
21 | } proc_info_t;
22 | #endif
23 |
24 | typedef int pipe_t;
25 | typedef int pid_t;
26 |
27 | int CreatePipe(pipe_t new_pipe[2]);
28 | pid_t CloneProcess();
29 | int Exec(const char* path, char* argv[]);
30 | int LinkFDtoIN(int fd);
31 | int LinkFDtoOUT(int fd);
32 | int OpenObject(const char* path, int flags, int mod);
33 | int WaitProcess();
34 | int ClosePipe(pipe_t pipe);
35 | int WritePipe(pipe_t pipe, void* buffer, int bytes);
36 | int ReadPipe(pipe_t pipe, void* buffer, int bytes);
37 |
38 | int CreateSharedMemory(const char* name, int size);
39 | int OpenSharedMemory(const char* name);
40 | void* MapMemory(int fd, int size);
41 | int UnmapMemory(void* addr, int size);
42 | int CloseSharedMemory(int fd);
43 | int UnlinkSharedMemory(const char* name);
```

Листинг 1: os.h

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include <unistd.h>
5 | #include <sys/wait.h>
6 | #include <sys/mman.h>
7 | #include <sys/stat.h>
8 | #include <fcntl.h>
9 |
```

```

10 | #include "os.h"
11 |
12 | int CreatePipe(pipe_t new_pipe[2]) { return pipe(new_pipe); }
13 | pid_t CloneProcess() { return fork(); }
14 | int Exec(const char* path, char* argv[]) { return execv(path, argv); }
15 | int LinkFDtoIN(int fd) { return dup2(fd, STDIN_FILENO); }
16 | int LinkFDtoOUT(int fd) { return dup2(fd, STDOUT_FILENO); }
17 | int OpenObject(const char* path, int flags, int mod) { return open(path, flags, mod);
    | }
18 | int WaitProcess() { return wait(NULL); }
19 | int ClosePipe(pipe_t pipe) { return close(pipe); }
20 | int WritePipe(pipe_t pipe, void* buffer, int bytes) { return write(pipe, buffer, bytes
    | ); }
21 | int ReadPipe(pipe_t pipe, void* buffer, int bytes) { return read(pipe, buffer, bytes);
    | }
22 |
23 |
24 | int CreateSharedMemory(const char* name, int size) {
25 |     int fd = shm_open(name, O_CREAT | O_RDWR, 0666);
26 |     if (fd != -1 && size > 0) {
27 |         ftruncate(fd, size);
28 |     }
29 |     return fd;
30 | }
31 |
32 | int OpenSharedMemory(const char* name) {
33 |     return shm_open(name, O_RDWR, 0666);
34 | }
35 |
36 | void* MapMemory(int fd, int size) {
37 |     return mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
38 | }
39 |
40 | int UnmapMemory(void* addr, int size) {
41 |     return munmap(addr, size);
42 | }
43 |
44 | int CloseSharedMemory(int fd) {
45 |     return close(fd);
46 | }
47 |
48 | int UnlinkSharedMemory(const char* name) {
49 |     return shm_unlink(name);
50 | }

```

Листинг 2: os_linux.c

```

1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include <sys/mman.h>
5 | #include <sys/stat.h>
6 | #include <fcntl.h>
7 | #include <unistd.h>
8 | #include <sched.h>
9 |
10 | #include "os.h"

```

```

11
12 #define BUFFERSIZE 256
13 #define SHARED_MEM_SIZE 4096
14
15 typedef struct {
16     char data[BUFFERSIZE];
17     int ready; // 0-, 1-, 2-
18 } shared_data_t;
19
20 int main() {
21     pid_t pid;
22     char filename[BUFFERSIZE];
23
24     int shm_fd = shm_open("/numbers_shm", O_CREAT | O_RDWR, 0666); //
25     if(shm_fd == -1) {
26         perror("shm_open error");
27         exit(1);
28     }
29
30     if(ftruncate(shm_fd, SHARED_MEM_SIZE) == -1) { //
31         perror("ftruncate error");
32         exit(1);
33     }
34
35     // ( )
36     shared_data_t* shared_data = mmap(NULL, SHARED_MEM_SIZE,
37                                       PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0); //
38
39     if(shared_data == MAP_FAILED) {
40         perror("mmap error");
41         exit(1);
42     }
43
44     memset(shared_data, 0, SHARED_MEM_SIZE); // ( )
45     shared_data->ready = 0;
46
47     printf(" : ");
48     if(!fgets(filename, BUFFERSIZE, stdin)) {
49         perror("fgets error");
50         exit(1);
51     }
52
53     filename[strcspn(filename, "\n")] = 0;
54
55     pid = CloneProcess();
56     if(pid == -1) {
57         perror("fork error");
58         exit(1);
59     }
60
61     if(pid == 0) {
62         if(Exec("./child", (char*[]){ "child", filename, "/numbers_shm", NULL}) == -1) {
63             perror("exec error");
64             exit(1);
65         }
66     } else {
67         printf(" , 'stop'\n");
68     }
69 }

```

```

68     while(1) {
69         char buffer[BUFFERSIZE];
70         printf("\n");
71         fflush(stdout);
72
73         if(!fgets(buffer, BUFFERSIZE, stdin)) {
74             perror("fgets error");
75             break;
76         }
77
78         buffer[strcspn(buffer, "\n")] = 0;
79
80         if(strcmp(buffer, "stop") == 0) {
81             strcpy(shared_data->data, "stop");
82             shared_data->ready = 2; //
83             break;
84         }
85
86         strncpy(shared_data->data, buffer, BUFFERSIZE - 1); //
87         shared_data->data[BUFFERSIZE - 1] = '\0';
88         shared_data->ready = 1; //
89
90         while(shared_data->ready == 1) {
91             sched_yield(); //
92         }
93
94         if(shared_data->ready == 2) {
95             break;
96         }
97     }
98
99     WaitProcess();
100
101     munmap(shared_data, SHARED_MEM_SIZE); //
102     close(shm_fd); //
103     shm_unlink("/numbers_shm"); //
104
105 }
106
107 return 0;
108 }

```

Листинг 3: parent.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <ctype.h>
5  #include <sys/mman.h>
6  #include <sys/stat.h>
7  #include <fcntl.h>
8  #include <unistd.h>
9  #include <sched.h>
10
11 #include "os.h"
12
13 #define BUFFERSIZE 256

```



```

14 #define SHARED_MEM_SIZE 4096
15
16 typedef struct {
17     char data[BUFFERSIZE];
18     int ready; // 0-, 1-, 2-
19 } shared_data_t;
20
21 int main(int argc, char* argv[]) {
22     char buffer[BUFFERSIZE];
23     FILE* file;
24
25     if(argc < 3) {
26         exit(1);
27     }
28
29     file = fopen(argv[1], "w");
30     if(file == NULL) {
31         perror("file open error");
32         exit(1);
33     }
34
35     int shm_fd = shm_open(argv[2], O_RDWR, 0666); //
36     if(shm_fd == -1) {
37         perror("shm_open error in child");
38         exit(1);
39     }
40
41     shared_data_t* shared_data = mmap(NULL, SHARED_MEM_SIZE,
42                                     PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
43
44     if(shared_data == MAP_FAILED) {
45         perror("mmap error in child");
46         exit(1);
47     }
48
49     fflush(file);
50
51     printf("child \n");
52
53     while(1) {
54         if(shared_data->ready == 1) {
55             strncpy(buffer, shared_data->data, BUFFERSIZE);
56             buffer[BUFFERSIZE - 1] = '\0';
57
58             if(strcmp(buffer, "stop") == 0) {
59                 shared_data->ready = 2;
60                 break;
61             }
62
63             int valid = 1;
64             for(int i = 0; buffer[i] != '\0'; i++) {
65                 if(!isdigit(buffer[i]) && !isspace(buffer[i]) &&
66                    buffer[i] != '-' && buffer[i] != '+') {
67                     valid = 0;
68                     break;
69                 }
70             }

```

```

71         if(valid && strlen(buffer) > 0) { //
72
73             char* token = strtok(buffer, " ");
74             int sum = 0;
75             int count = 0;
76             char numbers_list[BUFFERSIZE] = "";
77
78             while(token != NULL) {
79                 int num = atoi(token);
80                 sum += num;
81                 count++;
82
83                 if(strlen(numbers_list) > 0) {
84                     strcat(numbers_list, " ");
85                 }
86                 strcat(numbers_list, token);
87
88                 token = strtok(NULL, " ");
89             }
90
91             fprintf(file, ": %s : %d \n",
92                     numbers_list, sum);
93             fflush(file);
94
95             printf(": %d\n", sum);
96         } else {
97             fprintf(file, " : %s\n", buffer);
98             fflush(file);
99             printf(" : %s\n", buffer);
100         }
101
102         shared_data->ready = 0; //
103     } else if(shared_data->ready == 2) {
104         //
105         break;
106     }
107
108     sched_yield();
109 }
110
111 fclose(file);
112
113 munmap(shared_data, SHARED_MEM_SIZE);
114 close(shm_fd);
115
116 printf("child \n");
117 return 0;
118 }

```

Листинг 4: child.c

```

15997 execve("./parent",["./parent"],0x7ffe96635318 /* 49 vars */)
= 0
15997 brk(NULL) = 0x5ecd70f72000
15997
mmap(NULL,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0) =

```

0x71cedf4ba000

15997 access("/etc/ld.so.preload",R_OK) = -1 ENOENT (Нет такого
файла или каталога)

15997 openat(AT_FDCWD,"/etc/ld.so.cache",O_RDONLY|O_CLOEXEC) = 3

15997 fstat(3,{st_mode=S_IFREG|0644,st_size=56203,...}) = 0

15997 mmap(NULL,56203,PROT_READ,MAP_PRIVATE,3,0) = 0x71cedf4ac000

15997 close(3) = 0

15997

openat(AT_FDCWD,"/lib/x86_64-linux-gnu/libc.so.6",O_RDONLY|O_CLOEXEC) = 3

15997

read(3,"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"... ,832)
= 832

15997

pread64(3,"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,784,64)
= 784

15997 fstat(3,{st_mode=S_IFREG|0755,st_size=2125328,...}) = 0

15997

pread64(3,"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,784,64)
= 784

15997 mmap(NULL,2170256,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =

0x71cedf200000

15997

mmap(0x71cedf228000,1605632,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,
= 0x71cedf228000

15997

mmap(0x71cedf3b0000,323584,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x1b0000)
= 0x71cedf3b0000

15997

mmap(0x71cedf3ff000,24576,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,
= 0x71cedf3ff000

15997

mmap(0x71cedf405000,52624,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,-1,
= 0x71cedf405000

15997 close(3) = 0

15997

mmap(NULL,12288,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0) =
0x71cedf4a9000

15997 arch_prctl(ARCH_SET_FS,0x71cedf4a9740) = 0

15997 set_tid_address(0x71cedf4a9a10) = 15997

15997 set_robust_list(0x71cedf4a9a20,24) = 0

15997 rseq(0x71cedf4aa060,0x20,0,0x53053053) = 0

15997 mprotect(0x71cedf3ff000,16384,PROT_READ) = 0

15997 mprotect(0x5ecd36c35000,4096,PROT_READ) = 0

15997 mprotect(0x71cedf4f8000,8192,PROT_READ) = 0

15997

prlimit64(0,RLIMIT_STACK,NULL,{rlim_cur=8192*1024,rlim_max=RLIM64_INFINITY})
= 0

15997 munmap(0x71cedf4ac000,56203) = 0

```

15997
openat(AT_FDCWD, "/dev/shm/numbers_shm", O_RDWR|O_CREAT|O_NOFOLLOW|O_CLOEXEC, 0666)
= 3
15997 ftruncate(3, 4096) = 0
15997 mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) =
0x71cedf4b9000
15997 fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) =
0
15997 getrandom("\x24\xc6\xdf\x51\x03\x5f\xa1\x69", 8, GRND_NONBLOCK)
= 8
15997 brk(NULL) = 0x5ecd70f72000
15997 brk(0x5ecd70f93000) = 0x5ecd70f93000
15997 fstat(0, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) =
0
15997 write(1, "\320\262\320\265\320\264\320\270\321\202\320\265
\320\270\320\274\321\217 \321\204\320\260\320\271\320\273\320\260: ", 32)
= 32
15997 read(0, "1.txt\n", 1024) = 6
15997
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tid
= 16056
15997
write(1, "\320\262\320\262\320\265\320\264\320\270\321\202\320\265
\321\207\320\270\321\201\320\273\320\260, \320\264\320\273\321"... , 77)
<unfinished ...>
16056 set_robust_list(0x71cedf4a9a20, 24 <unfinished ...>)
15997 <... write resumed> = 77
16056 <... set_robust_list resumed> = 0
15997 write(1, "\n", 1) = 1
15997 read(0, <unfinished ...>)
16056
execve("./child", ["child", "1.txt", "/numbers_shm"], 0x7ffd2f3db418 /* 49
vars */) = 0
16056 brk(NULL) = 0x5e2610fd0000
16056
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7e8520494000
16056 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (Нет такого
файла или каталога)
16056 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
16056 fstat(3, {st_mode=S_IFREG|0644, st_size=56203, ...}) = 0
16056 mmap(NULL, 56203, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7e8520486000
16056 close(3) = 0
16056
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
16056
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"... , 832)
= 832

```

```

16056
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64)
= 784
16056 fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
16056
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64)
= 784
16056 mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7e8520200000
16056
mmap(0x7e8520228000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) =
0x7e8520228000
16056
mmap(0x7e85203b0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) =
0x7e85203b0000
16056
mmap(0x7e85203ff000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) =
0x7e85203ff000
16056
mmap(0x7e8520405000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) =
0x7e8520405000
16056 close(3) = 0
16056
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7e8520483000
16056 arch_prctl(ARCH_SET_FS, 0x7e8520483740) = 0
16056 set_tid_address(0x7e8520483a10) = 16056
16056 set_robust_list(0x7e8520483a20, 24) = 0
16056 rseq(0x7e8520484060, 0x20, 0, 0x53053053) = 0
16056 mprotect(0x7e85203ff000, 16384, PROT_READ) = 0
16056 mprotect(0x5e25f9033000, 4096, PROT_READ) = 0
16056 mprotect(0x7e85204d2000, 8192, PROT_READ) = 0
16056
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY})
= 0
16056 munmap(0x7e8520486000, 56203) = 0
16056 getrandom("\x45\x17\x56\x7c\xde\x7b\xbd\xc2", 8, GRND_NONBLOCK)
= 8
16056 brk(NULL) = 0x5e2610fd0000
16056 brk(0x5e2610ff1000) = 0x5e2610ff1000
16056 openat(AT_FDCWD, "1.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
16056
openat(AT_FDCWD, "/dev/shm/numbers_shm", O_RDWR|O_NOFOLLOW|O_CLOEXEC) = 4
16056 mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) =
0x7e8520493000
16056 fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) =
0
16056 write(1, "child

```

```

\320\267\320\260\320\277\321\203\321\211\320\265\320\275\n",21) = 21
15997 <... read resumed>"1 2 3 4 5 6 7\n",1024) = 14
16056 fstat(3,{st_mode=S_IFREG|0664,st_size=0,...}) = 0
16056 write(3,"\321\207\320\270\321\201\320\273\320\260: 1 2 3 4 5
6 7 \321\201\321\203"... ,44) = 44
16056 write(1,"\321\201\321\203\320\274\320\274\320\260: 28\n",15)
= 15
15997 write(1,"\n",1) = 1
15997 read(0,"1111 2222 333\n",1024) = 14
16056 write(3,"\321\207\320\270\321\201\320\273\320\260: 1111 2222
333 \321\201\321\203"... ,46) = 46
16056 write(1,"\321\201\321\203\320\274\320\274\320\260:
3666\n",17) = 17
15997 write(1,"\n",1) = 1
15997 read(0,"11111111 55555555\n",1024) = 18
16056 write(3,"\321\207\320\270\321\201\320\273\320\260: 11111111
55555555 "... ,54) = 54
16056 write(1,"\321\201\321\203\320\274\320\274\320\260:
66666666\n",21) = 21
15997 write(1,"\n",1) = 1
15997 read(0,"stop\n",1024) = 5
15997 wait4(-1, <unfinished ...>
16056 close(3) = 0
16056 munmap(0x7e8520493000,4096) = 0
16056 close(4) = 0
16056 write(1,"child
\320\267\320\260\320\262\320\265\321\200\321\210\320\265\320\275\n",23) =
23
16056 exit_group(0) = ?
16056 +++ exited with 0 +++
15997 <... wait4 resumed>NULL,0,NULL) = 16056
15997 ---SIGCHLD
{si_signo=SIGCHLD,si_code=CLD_EXITED,si_pid=16056,si_uid=1000,si_status=0,si_utime=53.
/* 53.49 s */ ,si_stime=0} ---
15997 munmap(0x71cedf4b9000,4096) = 0
15997 close(3) = 0
15997 unlink("/dev/shm/numbers_shm") = 0
15997 exit_group(0) = ?
15997 +++ exited with 0 +++

```

Листинг 5: *Strace логи*